Assignment – 13.5

Name:Sharwan
H.no: 2303A51957
Batch: 22

Task Description #1 (Refactoring – Removing Global Variables) •
Task: Use AI to eliminate unnecessary global variables from the code.
• Instructions:
o Identify global variables used across functions. o Refactor the code to pass values using function parameters. o Improve modularity and testability.
• Sample Legacy Code:
rate = 0.1 def calculate_interest(amount): return amount * rate print(calculate_interest(1000)) • Expected Output: o Refactored version passing rate as a parameter or using a configuration structure.

Prompt:
Refactor the Python code to remove global variables. Pass rate as a parameter and improve modularity and testability.

Code:

```
ai13.py > ...
'''Refactor the Python code to remove global variables. Pass rate as a parameter and improve modularity and testability.'''
def calculate_final_price(price, tax_rate):
    """Calculate the final price after applying tax.

    Args:
        price (float): The original price of the item.
        tax_rate (float): The tax rate to be applied (e.g., 0.07 for 7%).

    Returns:
        float: The final price after tax.
    """
    return price * (1 + tax_rate)
# Example usage:
if __name__ == "__main__":
    original_price = 100.0
    tax_rate = 0.07
    final_price = calculate_final_price(original_price, tax_rate)
    print(f"Original Price: ${original_price:.2f}")
    print(f"Tax Rate: {tax_rate * 100:.2f}%")
    print(f"Final Price: ${final_price:.2f}")
```

```
PROBLEMS 129   OUTPUT   DEBUG CONSOLE   TERMINAL 9   PORTS

PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Use
rs/2303A/OneDrive/Desktop/python course/ai13.py"
Original Price: $100.00
Tax Rate: 7.00%
Final Price: $107.00
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

- Global variable removed.
- Function is now reusable and testable.
- Follows modular programming principles.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

- Focus Areas: o Readability o Logical simplification o Maintainability Legacy Code: score = 78 if score >= 90: print("Excellent") else: if score >= 75: print("Very Good") else: if score >= 60: print("Good") else:

print("Needs Improvement") Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.

Prompt:

Simplify deeply nested if-else conditions to improve readability and maintainability. **Code:**

ai13.py > ...

```python
'''Simplify deeply nested if-else conditions to improve readability and maintainability to classify a number as positive, neg
def classify_number(num):
    """Classify a number as positive, negative, or zero.

    Args:
        num (int or float): The number to classify.

    Returns:
        str: The classification of the number ("positive", "negative", or "zero").
    """
    if num > 0:
        return "positive"
    elif num < 0:
        return "negative"
    else:
        return "zero"
# Example usage:
if __name__ == "__main__":
    numbers = [10, -5, 0]
    for number in numbers:
        classification = classify_number(number)
        print(f"The number {number} is classified as: {classification}")
```

PROBLEMS 130 OUTPUT DEBUG CONSOLE TERMINAL 9 PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Use
rs/2303A/OneDrive/Desktop/python course/ai13.py"
The number 10 is classified as: positive
The number -5 is classified as: negative
The number 0 is classified as: zero
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

- Removed nested structure.
- Improved readability.
- Easier to modify grade ranges.

Task 3 (Refactoring Repeated File Handling Code) • Task: Use AI to refactor repeated file open/read/close logic.

- Focus Areas: o DRY principle o Context managers

o Function reuse Legacy Code: f = open("data1.txt") print(f.read()) f.close() f = open("data2.txt") print(f.read()) f.close()

Expected Outcome:

o Reusable function using with open() and parameters.

Prompt:

Refactor repeated file open/close logic using context managers and reusable functions.

Code:

ai13.py > read_file

```python
''' Generate a python function to refactor repeated file open/close logic using context managers and reusable functions.'''
from pathlib import Path


def read_file(file_name):
    """Reads the content of a file and prints it.

    Args:
        file_name (str): The name of the file to be read.
    """
    file_path = Path(__file__).resolve().parent / file_name
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            print(f.read())
    except FileNotFoundError:
        print(f"File not found: {file_path}")


# Example usage:
if __name__ == "__main__":
    read_file("data1.txt")
    read_file("data2.txt")
```

PROBLEMS 127 OUTPUT DEBUG CONSOLE TERMINAL 9 PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\Activate.ps1"
& : File C:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\Activate.ps1 cannot be loaded because running scripts is
disabled on this system. For more information, see about_Execution_Policies at https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:3
+ & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\Activa ...
+   ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Use
rs/2303A/OneDrive/Desktop/python course/ai13.py"
File not found: C:\Users\2303A\OneDrive\Desktop\python course\data1.txt
File not found: C:\Users\2303A\OneDrive\Desktop\python course\data2.txt
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

- Applied DRY principle.
- Used with open() (context manager).
- Prevents file leaks.

Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.
- Focus Areas: o Time complexity o Data structure choice Legacy Code:

users = ["admin", "guest", "editor", "viewer"] name = input("Enter username: ") found = False for u in users: if u == name: found = True print("Access Granted" if found else "Access Denied") Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Prompt:

Replace inefficient linear search with optimized data structure and justify time complexity.

Code:

ai123.py > optimized_search

```python
'''Generate a python function to replace inefficient linear search with optimized data structure and justify time complexity.
def optimized_search(arr, target):
    """Search for a target value in a sorted list using binary search.

    Time Complexity: O(log n) - Binary search divides the search interval in half with each step, leading to logarithmic time
    Space Complexity: O(1) - Binary search is an in-place algorithm that does not require additional space.

    Args:
        arr (list): A sorted list of elements to search through.
        target: The value to search for.

    Returns:
        int: The index of the target value if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
# Example usage:
if __name__ == "__main__":
    sorted_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    target_value = 5
    result = optimized_search(sorted_list, target_value)
    if result != -1:
        print(f"Target {target_value} found at index: {result}")
    else:
        print(f"Target {target_value} not found in the list.")
```

PROBLEMS 136 OUTPUT DEBUG CONSOLE TERMINAL 9 PORTS

```
disabled on this system. For more information, see about_Execution_Policies at https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:3
+ & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\Activa ...
+   ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Use
rs/2303A/OneDrive/Desktop/python course/ai123.py"
Target 5 found at index: 4
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

- Changed list → set.
- Time complexity improved from **O(n)** to **O(1)** average case.

- More scalable for large datasets.

Task 5 (Refactoring Procedural Code into OOP Design) •
Task: Use AI to refactor procedural code into a class-based design.

- Focus Areas: o Object-Oriented principles o Encapsulation

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

Prompt:

Convert procedural salary calculation code into an object-oriented design with encapsulation.

Code:

ai123.py > ...

```python
'''Convert procedural salary calculation code into an object-oriented design with encapsulation.'''
class Employee:
    def __init__(self, name, base_salary):
        self.name = name
        self.base_salary = base_salary

    def calculate_salary(self):
        # Assuming some complex logic for salary calculation
        return self.base_salary * 1.2  # Example: adding a 20% bonus
# Example usage
if __name__ == "__main__":
    emp1 = Employee("Alice", 50000)
    emp2 = Employee("Bob", 60000)

    print(f"{emp1.name}'s salary: {emp1.calculate_salary()}")
    print(f"{emp2.name}'s salary: {emp2.calculate_salary()}")
```

PROBLEMS 133 OUTPUT DEBUG CONSOLE TERMINAL 9 PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Users/2303A/OneDrive/Desktop/python course/ai123.py"
Alice's salary: 60000.0
Bob's salary: 72000.0
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Observation:

- Applied OOP principles.
- Encapsulation achieved.
- Reusable for multiple employees.

6 (Refactoring for Performance Optimization)

Use AI to refactor a performance-heavy loop handling large data.

- Focus Areas: o Algorithmic optimization o Use of built-in functions Legacy Code: total = 0 for i in range(1, 1000000): if i % 2 == 0: total += i print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Prompt:

Optimize the loop summing even numbers using mathematical formulas or efficient Python constructs.

Code:

ai123.py > ...

```python
'''Optimize the loop summing even numbers using mathematical formulas or efficient Python constructs.'''
def sum_even_numbers(n):
    """Calculate the sum of even numbers from 1 to n using a mathematical formula.

    Time Complexity: O(1) - The formula allows us to calculate the sum in constant time.
    Space Complexity: O(1) - No additional space is used.

    Args:
        n (int): The upper limit of the range to sum even numbers.

    Returns:
        int: The sum of even numbers from 1 to n.
    """
    # The number of even numbers from 1 to n is n//2
    m = n // 2
    # The sum of the first m even numbers is m * (m + 1)
    return m * (m + 1)
# Example usage:
if __name__ == "__main__":
    n = 10
    print(f"The sum of even numbers from 1 to {n} is: {sum_even_numbers(n)}")  # Output: 30
```

PROBLEMS 138 OUTPUT DEBUG CONSOLE TERMINAL 9 PORTS

```
PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Users/2303A/OneDrive/Desktop/python course/ai123.py"
The sum of even numbers from 1 to 10 is: 30
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Task •

Task:

Observation:

- Replaced O(n) loop with O(1) formula.
- Massive performance improvement.
- Suitable for large-scale computations.

7 (Removing Hidden Side Effects)

Refactor code that modifies shared mutable state.

- Focus Areas: o Functional-

style refactoring

o Predictability Legacy Code: data = []

def add_item(x): data.append(x)

add_item(10) add_item(20) print(data)

Expected Outcome:

o Refactored function returning

values instead of mutating globals.

Prompt:

Refactor function to avoid modifying global mutable state and return new values instead.

Code:

```
ai123.py > ...
'''Refactor function to avoid modifying global mutable state and return new values instead.'''
def add_to_list(lst, item):
    """Add an item to a list and return the new list.

    Args:
        lst (list): The original list.
        item: The item to be added to the list.

    Returns:
        list: A new list with the item added.
    """
    new_lst = lst.copy()  # Create a copy of the original list to avoid modifying it
    new_lst.append(item)  # Add the item to the new list
    return new_lst  # Return the new list with the item added
# Example usage:
original_list = [1, 2, 3]
new_list = add_to_list(original_list, 4)
print("Original list:", original_list)  # Output: Original list: [1, 2, 3]
print("New list:", new_list)  # Output: New list: [1, 2, 3, 4]
```

```
PROBLEMS 134   OUTPUT   DEBUG CONSOLE   TERMINAL 9   PORTS

PS C:\Users\2303A\OneDrive\Desktop\python course> & "c:\Users\2303A\OneDrive\Desktop\python course\.venv\Scripts\python.exe" "c:/Use
rs/2303A/OneDrive/Desktop/python course/ai123.py"
Original list: [1, 2, 3]
New list: [1, 2, 3, 4]
PS C:\Users\2303A\OneDrive\Desktop\python course>
```

Task •

Task:

Observation:

- Removed global mutation.
- Function is now pure and predictable.
- Improves testability.

8 (Refactoring Complex Input Validation Logic)

Use AI to simplify and modularize complex validation rules.

• Focus Areas: o Readability o Testability Legacy Code: password = input("Enter password: ") if len(password) >= 8: if any(c.isdigit() for c in password): if any(c.isupper() for c in password): print("Valid Password") else: print("Must contain uppercase") else:
print("Must contain digit") else:
print("Password too short") Expected Outcome:
o Separate validation functions with clear responsibility **Prompt:**
Modularize password validation rules into separate functions to improve readability and testability. **Code:**

2.py

```python
'''Modularize password validation rules into separate functions to improve readability and testability.'''
def is_valid_length(password):
    return len(password) >= 8
def has_uppercase(password):
    return any(char.isupper() for char in password)
def has_lowercase(password):
    return any(char.islower() for char in password)
def has_digit(password):
    return any(char.isdigit() for char in password)
def has_special_char(password):
    special_characters = "!@#$%^&*()-_=+[]{}|;:'\",.<>?/"
    return any(char in special_characters for char in password)
def validate_password(password):
    if not is_valid_length(password):
        return "Password must be at least 8 characters long."
    if not has_uppercase(password):
        return "Password must contain at least one uppercase letter."
    if not has_lowercase(password):
        return "Password must contain at least one lowercase letter."
    if not has_digit(password):
        return "Password must contain at least one digit."
    if not has_special_char(password):
        return "Password must contain at least one special character."
    return "Password is valid."
# Example usage:
if __name__ == "__main__":
    password = input("Enter a password to validate: ")
    result = validate_password(password)
    print(result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
SHARWAN/OneDrive/Documents/ai.py/'''Generate a Hash Table class with inse.py"
Sharwan …
HARWAN/OneDrive/Documents/ai.py/2.py
Enter a password to validate: Sharwan@1234
Password is valid.
PS C:\Users\SHARWAN\OneDrive\Documents\ai.py> ^C
PS C:\Users\SHARWAN\OneDrive\Documents\ai.py>
```

Ln 30, Col 5

Observation:

- Broke validation into small reusable functions.
- Improved readability.
- Easier to unit test each rule independently.